

CLINICAL AI CONTROL PLANE

Technical Architecture

CAG-01 · Governance Gateway for Clinical AI Assistants

Prepared by Engineering · 2026-08-21

Clinical AI Platform Group

PHI PROTECTION · FULL AUDIT TRAIL · HIPAA ALIGNED

Clinical AI Control Plane: Architecture

Version 0.1.0 · Java 21 · Spring Boot 3.3 · PostgreSQL · Redis · Kafka

SECTION 01

Overview

The control plane is a governance gateway that sits between clinical AI assistants (billing, clinical, front office, patient, support) and the model layer. It provides the cross-cutting capabilities an assistant fleet needs in a healthcare setting: **protection of PHI, per-tenant cost and usage observability, configurable human review, provenance for every output, and a complete audit trail**.

The default posture is **autonomy with observability**. Human review and hard budgets are policies an operator turns on where risk justifies them, not fixed gates applied to every request. The model call itself is thin and sits behind a vendor-neutral provider seam; the engineering value is in the governance plane around it.

SECTION 02

Layered design

```
HTTP -> Controller (thin) -> SuggestionService -> ModelProvider seam -> Mock | Re
              JWT: tenant_id,        |                                    (plugga
              scope claims           +-> UsageMeteringService (Redis)
                                     +-> RedactionService (PHI)
                                     +-> GovernancePolicy (review?)
                                     +-> Repository (Postgres, Flyway)
                                     +-> AuditPublisher (Kafka) -> AuditConsumer
```

- **Controllers (thin):** extract `tenant_id` and actor from the verified JWT, validate the DTO, call the service, return the response DTO. No query building, no policy logic.
- **SuggestionService (single orchestrator):** owns the governed lifecycle. Every request passes through the same plane in the same order.
- **Provider seam:** `ModelProvider` is the only interface the model is called through. `MockModelProvider` is the default; `RemoteModelProvider` is the marked seam for a real client.
- **Audit is a separate consumer:** transitions are published to Kafka and persisted to a Postgres read model by `AuditConsumer`, decoupled from the write path.

SECTION 03

The governed pipeline

1. An assistant calls the gateway with a JWT carrying a `tenant_id` claim and scopes.
2. `UsageMeteringService.admit()` meters the request per tenant against a Redis counter. Monitor mode (default) records and warns without blocking; enforce mode throws `RateLimitExceededException` over budget.
3. `RedactionService` strips direct identifiers from the input **before any model call**. Redaction is always on; it is a data-protection concern, not a policy toggle.
4. The model is called through the provider seam with the redacted text. Token usage is recorded back to the meter.
5. The `Suggestion` is persisted with its grounding references and provenance (provider, tokens, actor).
6. `GovernancePolicy` decides the next state: auto-approve as actor `policy:auto` unless the assistant type appears in `REVIEW_REQUIRED_ASSISTANTS`, in which case the suggestion stays `PENDING` for an authorized reviewer.
7. Every transition, automated or human, publishes an `AuditEvent` to Kafka.

SECTION 04

Suggestion state machine

```
PENDING --approve(actor)--> APPROVED --markExecuted()--> EXECUTED
   |
   +----reject(actor, reason)--> REJECTED
```

Transitions are enforced inside the `Suggestion` entity: decisions are only legal from `PENDING`, and only an `APPROVED` suggestion can be executed. Invalid transitions throw `IllegalStateException` rather than silently correcting. The policy actor (`policy:auto`) and human actors travel the same transitions, so the audit trail is uniform across autonomous and reviewed paths.

SECTION 05

Multi-tenancy and security

- Stateless OAuth2 resource server; every `/api` request requires a JWT verified against the configured public key. Only actuator health/info, the OpenAPI UI, and `/dev/**` are open.
- **Tenant isolation at the query:** the `tenant_id` claim is mandatory, and every repository read filters by it (`findByIdAndTenantId`). A tenant can never address another tenant's suggestion id.
- **Scope-gated operations:** `suggestion:create`, `suggestion:approve` (approve and reject), and `suggestion:execute` are independent authorities checked with method-level `@PreAuthorize`.
- The `dev` profile enables a local token minting endpoint (`GET /dev/token`) signed by a throwaway RSA key pair used only for local development and tests. Production validates tokens from a real identity provider.

SECTION 06

Audit trail

Every lifecycle event (`SUGGESTION_CREATED`, `SUGGESTION_AUTO_APPROVED`, `SUGGESTION_APPROVED`, `SUGGESTION_REJECTED`, `SUGGESTION_EXECUTED`) is published to the `clinical-ai.audit` topic and persisted by a separate consumer into a Postgres read model, queryable per tenant and per suggestion.

Publishing currently happens inside the write transaction for clarity. The dual-write between the database and Kafka is a known seam; the production-grade answer is a **transactional outbox**, recorded in ADR 0003.

SECTION 07

API contract

ENDPOINT	SCOPE	PURPOSE
`POST /api/v1/suggestions`	`suggestion:create`	Run the governed pipeline; returns the suggestion with status and provenance.
`GET /api/v1/suggestions`, `GET /api/v1/suggestions/{id}`	authenticated	List and read, always tenant-filtered.
`POST /api/v1/suggestions/{id}/approve`, `/reject`	`suggestion:approve`	Human decision from `PENDING`.
`POST /api/v1/suggestions/{id}/execute`	`suggestion:execute`	Mark an approved suggestion executed downstream.
`GET /api/v1/usage`	authenticated	Per-tenant usage counters for cost observability.
`GET /api/v1/audit`, `GET /api/v1/suggestions/{id}/audit`	authenticated	Audit read model, per tenant and per suggestion.

The contract is described by springdoc OpenAPI and served at `/swagger-ui.html`. Errors return a structured `ApiError` body via the global exception handler.

SECTION 08

Configuration surface

VARIABLE	DEFAULT	PURPOSE
`MODEL_PROVIDER`	`mock`	Model backend behind the seam: `mock` for local and tests, `remote` for a real backend.
`USAGE_MODE`	`monitor`	`monitor` records without blocking; `enforce` applies the hard budget.
`USAGE_RPM`	`600`	Per-tenant request budget for the soft warning and enforce mode.
`REVIEW_REQUIRED_ASSISTANTS`	empty	Comma-separated assistant types requiring human review. Empty means autonomy.

SECTION 09

Deployment topology

- **Container:** multi-stage Docker build (`docker/Dockerfile`); CI builds the image on every push.
- **One chart, two targets:** the Helm chart under `helm/clinical-ai-gateway` deploys both the local minikube proof (`deploy/local/up.sh` , pinned to the `minikube` context) and the EKS production target.
- **Infrastructure as code:** Terraform provisions the VPC and EKS cluster (`terraform/`).
- **CI gates:** build and Testcontainers-backed tests, Spotless formatting as a separate job, Trivy scan (vuln, secret, misconfig), and a Docker image build.

SECTION 10

Testing strategy

- **Unit:** redaction patterns, metering in monitor and enforce modes, and orchestration across the autonomous and review-required paths (Mockito).
- **Integration:** Testcontainers starts Postgres, Kafka, and Redis; the test drives the create-and-approve flow over HTTP with a signed token and awaits the audit consumer persisting the trail (Awaitility).

SECTION 11

Scope, dependencies, and assumptions

- The model call is mocked by default; the remote provider is a stub marking where a real client belongs.
- Redaction is pattern-based and illustrative; a production deployment would use a clinical NLP de-identification service behind the same boundary.
- PHI redaction and the audit trail are **always on**; they are data-protection and compliance concerns, not limits on what the AI can do.
- Schema changes go through Flyway migrations only (`ddl-auto: none`).
- Design rationale is recorded in ADRs 0001 through 0006 (companion document CAG-03).